Gomoku Game-Playing Agent Using Minimax and Alpha-Beta Pruning

Yaru Gao
Zeyuan Zhang
Md Borhan Uddin
Gregory Shanygin

1. Introduction

Gomoku is a classic two-player board game that presents a challenging problem in adversarial decision-making due to its large search space and complex tactical structure. Although the rules are simple, the combinatorial explosion of possible board configurations makes exhaustive search impractical, especially in mid- to late-game scenarios. As a result, effective gameplay requires a combination of strategic search and heuristic evaluation.

In this project, we develop a Gomoku game-playing agent based on classical artificial intelligence techniques. Specifically, we employ depth-limited minimax search enhanced with alpha-beta pruning to efficiently explore the game tree. To compensate for the inability to search to terminal states, we design a pattern-based heuristic evaluation function that captures key tactical features such as threats, opportunities, and positional advantages.

This project covers three main areas. First, we implement a complete adversarial search pipeline, including efficient move generation, pattern-based heuristic evaluation, and alpha-beta pruning with move ordering. Second, we test the agent against a random opponent, measuring decision time at search depths 1–3, win rate over 100 games, and average game length. Third, we compare plain minimax and alpha-beta pruning at equal depths and run a full 3×3 round-robin depth-matrix tournament to examine how search depth affects gameplay.
Experimental evaluation shows that alpha-beta pruning dramatically reduces the search cost over plain minimax, and that the agent plays effectively against a random baseline across all tested depths. The depth-matrix tournament further reveals that deeper search generally improves play quality, though a horizon effect can cause deeper agents to lose to shallower opponents in specific tactical configurations.

2. Problem Formulation

Gomoku is formulated as a two-player, deterministic, perfect-information, zero-sum game played on a 15 × 15 grid. Two players, denoted as BLACK and WHITE, alternately place stones on empty cells of the board. The objective is to be the first to form a sequence of five consecutive stones in a horizontal, vertical, or diagonal direction.
We model the game as a sequential decision-making problem. Let s ∈ S denote a game state, where S is the set of all possible board configurations. Each state s is represented by a matrix

$$s \in \{0, 1, 2\}^{15 \times 15}$$

where 0 denotes an empty cell, 1 denotes a black stone, and 2 denotes a white stone.
At any state s, the set of legal actions A(s) consists of all empty positions on the board. An action a ∈ A(s) corresponds to placing a stone at a specific location, resulting in a transition to a
new state s′ = T (s, a), where T is the deterministic transition function.
The game is zero-sum, meaning that the gain of one player corresponds to the loss of the other. We define a value function V(s) that measures the desirability of a state from the perspective of a given player. Terminal states correspond to either a win (five-in-a-row) or a draw (full board). The value function satisfies:

$$V(s) = \begin{cases} 1 & \text{if the player wins} \\ -1 & \text{if the opponent wins} \\ 0 & \text{if the game ends in a draw} \end{cases}$$

The objective of the agent is to select actions that maximize the expected value of the game outcome under optimal play. This can be formalized using the minimax principle:

$$\max_{a_1} \min_{a_2} \max_{a_3} \cdots V(s)$$

where players alternate between maximizing and minimizing the value function.
Due to the exponential growth of the game tree, exhaustive search to terminal states is computationally infeasible. Therefore, in practice, the value function V (s) is approximated using a heuristic evaluation function, and the search is truncated to a fixed depth. The goal of the agent is thus to approximate the optimal policy under these computational constraints.

3. Methodology

3.1 Game Representation

The Gomoku board is represented as a discrete 15 × 15 grid. Each cell in the grid can take one of three possible values:

0 (empty), 1 (black stone), 2 (white stone).

Formally, a game state s is defined as

$$s \in \{0, 1, 2\}^{15 \times 15}$$

where $s_{i,j}$ denotes the content of the cell at row i and column j.
Two players, denoted as BLACK and WHITE, take turns placing stones on empty cells of the board. By convention, the black player moves first. A move is represented as a tuple (i, j) corresponding to a valid empty position on the grid.
The state of the game evolves deterministically according to a transition function T (s, a), which updates the board configuration by placing the current player's stone at the selected position. A move is considered valid if and only if the selected cell is within the bounds of the board and currently unoccupied. A terminal state occurs when either player forms a sequence of five or more consecutive stones in a straight line. Winning configurations are detected along four principal directions:
• Horizontal,
• Vertical,

• Main diagonal (top-left to bottom-right),
• Anti-diagonal (top-right to bottom-left).
Additionally, the game terminates in a draw if all cells on the board are occupied and no player has achieved a five-in-a-row configuration.
This representation provides a structured and efficient way to model the game state, enabling systematic exploration of possible moves and facilitating the application of adversarial search algorithms.
The overall game flow of the system, covering both Player vs. AI and AI vs. AI benchmark modes, is illustrated in Figure 1 (see Appendix A).

3.2 Move Generation

A key challenge in designing an efficient Gomoku agent is the extremely large branching factor of the game. In a 15 × 15 board, there are up to 225 possible moves at the beginning of the game. Exhaustively considering all legal actions at each step would result in exponential growth of the search tree, making naive minimax search computationally infeasible.
To address this issue, we employ a constrained move generation strategy that significantly reduces the effective action space while preserving strategically relevant moves. Instead of considering all empty cells, we restrict candidate moves to positions within a fixed neighborhood of existing stones.
Formally, let $\mathcal{O}(s)$ denote the set of occupied cells in state s. For each $(i, j) \in \mathcal{O}(s)$, we consider all empty cells (i', j') such that

$$|i' - i| \leq r, \quad |j' - j| \leq r$$

where r is a predefined neighborhood radius. In our implementation, we set $r = 2$, resulting in a local 5×5 neighborhood around each occupied position. The union of all such neighborhoods defines the set of candidate moves:

$$\mathcal{A}_{\text{cand}}(s) = \{(i', j') \in \mathcal{A}(s) \mid \exists (i, j) \in \mathcal{O}(s) \text{ such that } |i' - i| \leq r, |j' - j| \leq r\}$$

If the board is empty, the agent selects the center position as the initial move, as it provides maximal flexibility for future expansions.

This localized move generation strategy is motivated by the observation that optimal moves in Gomoku typically occur near existing stones, where interactions and threats develop. By focusing only on these regions, we reduce the branching factor from $O(n^2)$ to a much smaller subset, enabling deeper search within practical time constraints.

To further improve efficiency, candidate moves are sorted based on their proximity to the center of the board. Moves closer to the center are prioritized, as they tend to offer greater strategic potential. This ordering is particularly beneficial for alpha-beta pruning, as exploring promising moves earlier increases the likelihood of pruning suboptimal branches.

Finally, to maintain computational tractability, we impose a fixed upper bound on the number of candidate moves evaluated at each search node. This additional pruning ensures that the search remains efficient even in mid-game positions with many nearby empty cells.

Overall, the proposed move generation mechanism provides an effective balance between computational efficiency and strategic completeness, forming a critical component of the overall search framework.

3.3 Heuristic Evaluation

Since exhaustive search of the Gomoku game tree is computationally infeasible, we approximate the true value function using a heuristic evaluation function. This function assigns a numerical score to non-terminal board states, reflecting their desirability from the perspective of a given player. The heuristic serves as a surrogate for the true minimax value at leaf nodes of the truncated search tree.

We model the evaluation function as a linear combination of pattern-based features. Let $\phi_i(s)$ denote the count of a specific pattern type i in state s, such as sequences of consecutive stones or threat configurations. The heuristic evaluation function is defined as

$$V(s) = \sum_i w_i \, \phi_i(s)$$

where w_i are weights assigned to each pattern type. This formulation can be interpreted as a feature-based approximation of the value function.

To efficiently identify patterns, the board is decomposed into all possible one-dimensional lines, including rows, columns, and both diagonal directions. Each line is encoded into a symbolic string representation, where:

x = current player's stone, o = opponent's stone or boundary, _ = empty cell.

This encoding allows pattern matching to be performed using string operations, enabling efficient detection of relevant configurations.

We consider a set of standard Gomoku patterns that capture varying levels of strategic importance:

• Five in a row: A winning configuration.

• Open four (*_xxxx_*): A configuration with two open ends, representing an immediate winning threat.

• Rush four: A configuration that can be completed to five in one move.

• Open three: A configuration that can be extended to an open four.

• Sleep three: A weaker three-in-a-row with limited extension potential.

• Open two / Sleep two: Early-stage patterns with potential for future growth.

Each pattern type is assigned a weight reflecting its strategic significance. Stronger patterns, such as open fours, are given significantly higher weights than weaker patterns, ensuring that the evaluation prioritizes imminent threats and winning opportunities.

Beyond individual patterns, we incorporate additional bonuses for combined threat configurations. These include:

• Multiple simultaneous threats (e.g., two open threes),

• Four-three combinations (a rush four and an open three),

• Multiple forcing moves that cannot be blocked simultaneously.

Such configurations are critical in Gomoku, as they often lead to forced wins. Incorporating these bonuses allows the heuristic to capture higher-order tactical interactions that are not represented by individual pattern counts alone.

The final evaluation score is computed as the difference between the player's score and the opponent's score:

$$\mathrm{Eval}(s) = V_{\mathrm{player}}(s) - \lambda\, V_{\mathrm{opponent}}(s)$$

where $\lambda > 1$ is a weighting factor that slightly amplifies the opponent's threats. In our implementation, this asymmetry encourages defensive play by prioritizing the blocking of opponent threats when necessary.

The proposed heuristic evaluation function provides a balance between computational efficiency and strategic expressiveness. By combining pattern-based features with weighted scoring and threat bonuses, the agent is able to approximate the true value of complex board states without exhaustive search. However, the approach relies on manually designed features and fixed weights, which may limit adaptability across different game scenarios. Future improvements could involve learning the feature weights from data or incorporating probabilistic models to better estimate win likelihoods.

3.4 Minimax and Alpha-Beta Pruning

To determine optimal actions under adversarial conditions, we employ the minimax algorithm, a standard approach for two-player zero-sum games. The core idea is to simulate all possible future moves up to a fixed depth, assuming that the agent seeks to maximize its advantage while the opponent seeks to minimize it.

Let $V(s)$ denote the value of a game state s from the perspective of the agent. The minimax recursion is defined as:

$$V(s) = \begin{cases} \max_{a \in \mathcal{A}(s)} V(T(s,a)), & \text{if it is the agent's turn,} \\ \min_{a \in \mathcal{A}(s)} V(T(s,a)), & \text{if it is the opponent's turn} \end{cases}$$

Here, $\mathcal{A}(s)$ is the set of legal actions and $T(s,a)$ is the deterministic transition function. Since the full game tree is too large to explore exhaustively, we apply depth-limited search and use the heuristic evaluation function at leaf nodes.

In practice, the recursion is truncated at a fixed depth d. At depth 0, or when a terminal state is reached, the value of the state is approximated using the heuristic function:

$$V(s) \approx \mathrm{Eval}(s)$$

This approximation allows the agent to evaluate non-terminal positions efficiently while maintaining a tractable search space.

To improve computational efficiency, we incorporate alpha-beta pruning into the minimax search. Alpha (α) represents the best score that the maximizing player can guarantee so far, while beta (β) represents the best score that the minimizing player can guarantee. During the search, branches that cannot influence the final decision are pruned according to the condition:

$$\beta \leq \alpha$$

When this condition is satisfied, further exploration of the current branch is unnecessary, as the opponent would avoid reaching such states.

The effectiveness of alpha-beta pruning depends heavily on the order in which moves are explored. To maximize pruning efficiency, candidate moves are prioritized based on their strategic importance. Specifically, moves that result in immediate wins or block the opponent's immediate winning threats are evaluated first. This ordering increases the likelihood of early pruning and reduces the overall number of nodes explored.
Even after restricting moves through local candidate generation, the branching factor may remain large in mid-game states. To address this, we impose an upper bound on the number of candidate moves evaluated at each node. This constraint ensures that the search remains computationally feasible while still considering the most promising actions.

During search, special handling is applied to terminal or near-terminal states. If a move results in an immediate win, the search terminates early and assigns a large positive score. Similarly, if the opponent can force a win, a large negative score is returned. Additionally, we incorporate a depth-based adjustment to prefer faster wins and delay losses.

The combination of depth-limited minimax and alpha-beta pruning provides an effective framework for adversarial decision-making in Gomoku. While the algorithm does not guarantee optimal play due to heuristic approximation and depth limitations, it enables the agent to explore strategically relevant portions of the game tree efficiently. The overall performance of the system is strongly influenced by the quality of the heuristic function and the effectiveness of move ordering.

3.5 AI Decision-Making Workflow

To provide a concrete view of how the agent operates at runtime, we describe the complete decision-making pipeline executed on every AI turn. The pipeline consists of four tightly coupled stages: (1) candidate move generation, (2) move ordering, (3) recursive alpha-beta search, and (4) heuristic evaluation at leaf nodes. Figure 2 summarises the complete pipeline.

Stage 1 — Candidate Move Generation. The agent restricts its search to empty cells within a Chebyshev-distance radius of 2 around every occupied stone, implemented in get_candidate_moves() (move_gen.py). This reduces the effective branching factor from up to 225 (full board) to a compact local neighbourhood. If the board is empty, the centre cell is returned as the sole candidate. The candidate set is shown in Figure 2 (Appendix B).

Stage 2 — Move Ordering. Before the recursive search begins, _order_moves() (search.py) sorts candidates in descending priority so the most promising moves are explored first, maximising the probability of early alpha-beta cut-offs. The branching factor is then capped at MAX_CANDIDATES = 15. The priority table is shown in Figure 3 (Appendix C).

Stage 3 — Recursive Alpha-Beta Search. alphabeta() (search.py) implements depth-limited minimax with alpha-beta pruning. α tracks the best score the maximiser can guarantee; β tracks the best score the minimiser can guarantee. When β ≤ α the remaining sibling nodes are pruned. On a cut-off, the responsible move is stored as a killer move and its history score is incremented by depth². Before recursing deeper, each candidate move is tested for an immediate win; if found, the search returns WIN_SCORE + depth without further expansion. A transposition table keyed on the board's Zobrist hash avoids re-expanding previously evaluated positions. The search tree structure is illustrated in Figure 4 (Appendix D).

Stage 4 — Heuristic Evaluation at Leaf Nodes. When the search reaches the configured depth limit or a terminal state, evaluate() (heuristic.py) is invoked. It scans all rows, columns, and diagonals; encodes each line as a symbolic string (x = current player, o = opponent/boundary, _ = empty); matches it against a pattern table; and computes Score = score(AI) − 1.1 × score(opponent). Combination-threat bonuses are added for configurations such as a four-three kill (+50 000) or a double open-three (+10 000) that cannot be simultaneously blocked. The final score propagates back up the search tree to guide the root decision. Pattern weights are listed in Figure 5 (Appendix E).

Starting from the root, the agent executes four stages on every turn (see Figure 6 in Appendix F for the full workflow diagram): (Stage 1) candidate moves are generated from empty cells within a radius-2 neighbourhood of existing stones; (Stage 2) candidates are sorted by priority — immediate winning moves, blocking moves, transposition-table move, killer moves, then history-heuristic score; (Stage 3) alpha-beta pruning is applied to skip branches that cannot improve the current best outcome, with cut-off moves stored as killers and their history score updated; (Stage 4) when the depth limit is reached, the heuristic evaluation function scans four directions to detect patterns (Five, Open-Four, Rush-Four, Open-Three, etc.) and returns Score = score(AI) − 1.1 × score(opponent). The best move found at the root is then placed on the board.

4. Experiments

We evaluate the performance of the Gomoku agent through three complementary experiments. The first experiment assesses the agent's basic competence by measuring win rate, game length, and decision time against a non-strategic random opponent. The second experiment directly compares plain minimax search with alpha-beta pruning at equal depths to quantify the efficiency gains of pruning. The third experiment evaluates how search depth affects gameplay quality through a comprehensive AI vs. AI depth-matrix tournament. All experiments are conducted with fixed random seeds to ensure reproducibility.

4.1 AI vs. Random Player

This experiment evaluates the agent's effectiveness against a non-strategic baseline and characterises its computational cost across search depths.
Setup. The agent plays as Black using alpha-beta search. The opponent plays as White by selecting moves uniformly at random from the candidate-move set. A fixed random seed (seed = 42) is applied throughout to ensure reproducibility. Decision time is measured on a fixed mid-game board configuration; the search is repeated five times at each depth and the average is reported. Win rate and game length are evaluated over 100 complete games at search depth 2.
Metrics. Three metrics are collected: (1) decision time — average time per move at depths 1, 2, and 3; (2) win rate — the number of wins, losses, and draws over 100 games; (3) game length — average number of moves per game. Together these metrics assess both computational efficiency and gameplay strength.

4.2 Minimax vs. Alpha-Beta Pruning

This experiment measures the efficiency gains of alpha-beta pruning by benchmarking it against plain minimax at identical search depths.
Setup. One side uses plain minimax with no pruning or auxiliary data structures; the other uses alpha-beta search enhanced with a transposition table, killer heuristic, and history heuristic. Both sides search to the same depth. Three games are run per depth level across depths 1 to 3, yielding 9 games in total. To remove first-mover bias, the minimax player alternates colours across games — Black in odd-numbered games, White in even-numbered games. Each game starts from a distinct, randomly seeded opening stone placed in the centre 5×5 region of the board. The transposition table, killer moves, and history heuristic are reset before each game to ensure clean measurements.
Metrics. For each algorithm the following efficiency metrics are recorded per move: (1) average and maximum nodes explored; (2) pruning rate — fraction of pruned nodes relative to all nodes explored plus pruned nodes; (3) effective branching factor (EBF), computed as (total nodes / num_moves)^(1⁄depth); (4) node reduction (%) — percentage decrease in nodes relative to minimax; (5) time speedup factor — minimax average time divided by alpha-beta average time.

4.3 Depth Analysis

This experiment examines the effect of search depth on gameplay quality through a full round-robin AI vs. AI tournament.
Setup. A complete 3×3 matchup matrix is run for all pairings of depths drawn from the set {1, 2, 3}. Three games are played per matchup, yielding 27 games in total. Both sides use the same alpha-beta search implementation, heuristic evaluation function, and move-ordering strategy; only the search depth differs. For each game, a randomly seeded opening stone is placed in the centre 5×5 region to prevent deterministic repetition, and the game state (transposition table, killer moves, history heuristic) is reset to ensure independent measurements.

Metrics. The primary metric is the win rate of each side across the three games in each matchup. Secondary metrics include total moves per game, average decision time, and per-depth search efficiency (average nodes, pruning rate, EBF), which contextualise the win-rate differences in terms of computational cost.

5. Results and Analysis

We present the results of the three experiments described in Section 4, followed by analysis and key insights. All experiments were run with a fixed random seed (seed = 42 for the random-opponent games; per-game seeds for the AI vs. AI benchmarks) to ensure reproducibility.

5.1 AI vs. Random Player

Tables 1–3 (see Appendices K–M) summarise the results. Table 1 reports average decision time per move at search depths 1, 2, and 3, measured on a fixed mid-game board configuration over five repetitions per depth. Table 2 shows the win / loss / draw record over 100 games at depth 2 against a uniformly random opponent. Table 3 reports the average game length over the same 100 games. Figure 7 (see Appendix G) plots the decision-time trend across depths.

Depth	Avg. Decision Time (s)
1	0.0019
2	0.0100
3	0.0253

Table 1: Average decision time per move at search depths 1–3.
Table 2: Win / loss / draw record over 100 games at search depth 2 against a uniformly random opponent.

Wins	Losses	Draws	Win Rate
100	0	0	100%

Table 3: Average game length over the same 100 games.

Avg. Moves per Game
13.2

Win rate and game length. The agent wins every game (100%) and terminates in an average of only 13.2 moves. This demonstrates that the pattern-based heuristic reliably identifies and exploits tactical opportunities early, directing the search toward winning patterns such as open fours and four-three combinations well before the random opponent can mount a defence.

Decision time. Time scales roughly 5× per depth increment: depth 1 → depth 2 is a 5.3× increase (0.0019 s to 0.0100 s), and depth 1 → depth 3 is a 13.3× increase (0.0019 s to 0.0253 s). At depth 2, the agent responds in under 11 ms per move, which is well within

real-time play requirements. Even at depth 3, the 25 ms response time is comfortable for interactive use.

5.2 Minimax vs. Alpha-Beta Pruning

Table 4 compares plain minimax and alpha-beta search at depths 1–3, aggregated over three games per depth.
Table 4: Efficiency comparison of plain minimax and alpha-beta search at equal depths (3 games per depth, moves aggregated across all games).

Depth	MM Avg Nodes	AB Avg Nodes	Node Reduction	AB Pruning Rate	EBF (MM)	EBF (AB)	Speedup
1	13	16	−16.2 %	0.0 %	12.79	16.00	0.78×
2	211	78	+63.5 %	12.5 %	14.53	8.82	2.77×
3	3006	326	+89.7 %	8.6 %	14.43	6.88	13.03×

Depth 1 — no benefit. At depth 1, alpha-beta produces no pruning (rate = 0.0 %) and is actually slightly slower than minimax (speedup 0.78×). The search tree is too shallow for the $\beta \leq \alpha$ condition to trigger, while the overhead of maintaining the transposition table and killer / history tables adds a small constant cost. This is consistent with the theoretical requirement that alpha-beta needs at least a two-ply tree to prune any branch.

Depth 2 — clear advantage. Alpha-beta eliminates 63.5 % of nodes and achieves a 2.77× speedup. The EBF drops from 14.53 (minimax) to 8.82 (alpha-beta), meaning the alpha-beta search tree is equivalent in cost to a minimax tree of branching factor ≈8.82 — substantially shallower. The 12.5 % pruning rate shows that move ordering (Stage 2, Section 3.5) already directs the search toward high-quality moves that generate useful α and β bounds.

Depth 3 — dramatic gain. The advantage of alpha-beta becomes dramatic: 89.7 % node reduction and a 13.03× speedup. The EBF falls from 14.43 to 6.88, approaching the theoretical ideal of O(b^{d/2}) nodes for a perfectly ordered search. This confirms that move ordering — placing immediate wins, blocks, transposition-table moves, and killer moves first — is the primary driver of pruning efficiency. As depth increases, early cut-offs cascade through more levels of the tree, compounding the savings.

5.3 Depth Analysis

Figure 9 (see Appendix I) shows the win-rate heatmap for all nine Black-vs-White depth pairings from the 3×3 round-robin tournament (3 games per matchup, 27 games total). Table 6 shows the depth-scaling curve for the symmetric (same-depth) diagonal, and Figure 10 (see Appendix J) plots the corresponding node-count and decision-time trends.
Table 6: Depth-scaling curve for same-depth mirror matches (Black perspective, alpha-beta search).

Depth	Avg Nodes	Avg Time (s)	Pruning Rate	Avg EBF
1	14	0.0084	0.0 %	13.69
2	58	0.0315	17.3 %	7.60
3	295	0.1180	8.8 %	6.66

General depth advantage. Depth 2 dominates depth 1 decisively (Black wins 100 % in both colour roles), and depth 3 beats depth 2 in the majority of matchups (66.7 %). This confirms that deeper search translates to stronger play, as the agent can see further into the future and anticipate multi-move threats that shallower search misses.

Horizon effect. The most counter-intuitive result is that depth-1 Black defeats depth-3 White with a 66.7 % win rate. This is an instance of the horizon effect: the depth-3 agent's search horizon occasionally falls just short of seeing the tactical threats that a depth-1 player exploits through immediate, reactive moves. The depth-3 agent invests computation in long, narrow lines while sometimes failing to prevent a shallow forced win that depth 1 executes in a single move. The effect is amplified by the small sample size (3 games) and randomised openings, which can place both agents in positions where short-term tactics dominate.

Depth-scaling cost and pruning. Node count scales approximately 4–5× per depth step (14 → 58 → 295), and wall-clock time scales similarly (0.0084 s → 0.0315 s → 0.1180 s). The EBF decreases from 13.69 at depth 1 to 7.60 at depth 2 and 6.66 at depth 3, reflecting that pruning and move ordering become progressively more effective as more cut-off opportunities arise deeper in the tree. Notably, the pruning rate peaks at depth 2 (17.3 %) and dips slightly at depth 3 (8.8 %), suggesting that at depth 3 the board position has diversified enough that fewer sibling branches are dominated by a single line.

First-mover advantage. In symmetric same-depth mirror matches, Black wins more often than White at every depth (66.7 % at depths 1 and 3; 100 % at depth 2), reflecting the inherent first-mover advantage in Gomoku. Placing the first stone allows Black to establish central control, forcing White into a reactive position throughout the game.

6. Conclusion

Experimental results demonstrate that the agent achieves a 100 % win rate at depth 2 over 100 games against a random opponent, with an average game length of 13.2 moves and a per-move decision time of 10 ms. Decision time scales approximately 2.5–3× per depth step (1.9 ms → 10.0 ms → 25.3 ms), confirming the inherent trade-off between search depth and computational cost.

Our algorithmic analysis shows that alpha-beta pruning reduces node expansions by up to 89.7 % and achieves a 13.03× speedup over plain minimax at depth 3. The effective branching factor decreases from 13.69 at depth 1 to 6.66 at depth 3, reflecting that pruning and move ordering become progressively more effective as deeper search creates more cut-off

opportunities. These results confirm the importance of pruning techniques in making adversarial search tractable at practical depths.

The depth-matrix tournament reveals that while depth 2 dominates depth 1 decisively (100 % win rate in both colour roles), a horizon effect causes depth-1 Black to defeat depth-3 White in 66.7 % of games: the deeper agent's search horizon falls just short of detecting shallow forced wins that depth 1 executes immediately. Together, these results show that raw depth does not monotonically translate to stronger play, and that heuristic quality and tactical awareness matter as much as search depth.

Future work could explore learning-based weight tuning, quiescence search to mitigate the horizon effect, and iterative deepening or Monte Carlo tree search to improve the depth–cost trade-off.

Overall, this project shows that classical AI search techniques, when carefully implemented and combined, can yield an effective and computationally efficient Gomoku-playing agent.

7. Additional Links

GitHub Repository Link: https://github.com/YaruG1022/Gomoku-Agent.git

Colab Demo Notebook Link:
https://drive.google.com/file/d/1wT8iw3VW2vC7e0CZeQa4OeKK7YE-5fu4/view?usp=sharing

Presentation Slides Link:
https://1drv.ms/p/c/276a7883772755be/IQDzDrHGPNhaRbJv226v-izlAYfgGL2gerT9F2adehzcBug?e=8he1bh

Appendix

Appendix A: Figure 1 – Overall Game Flow

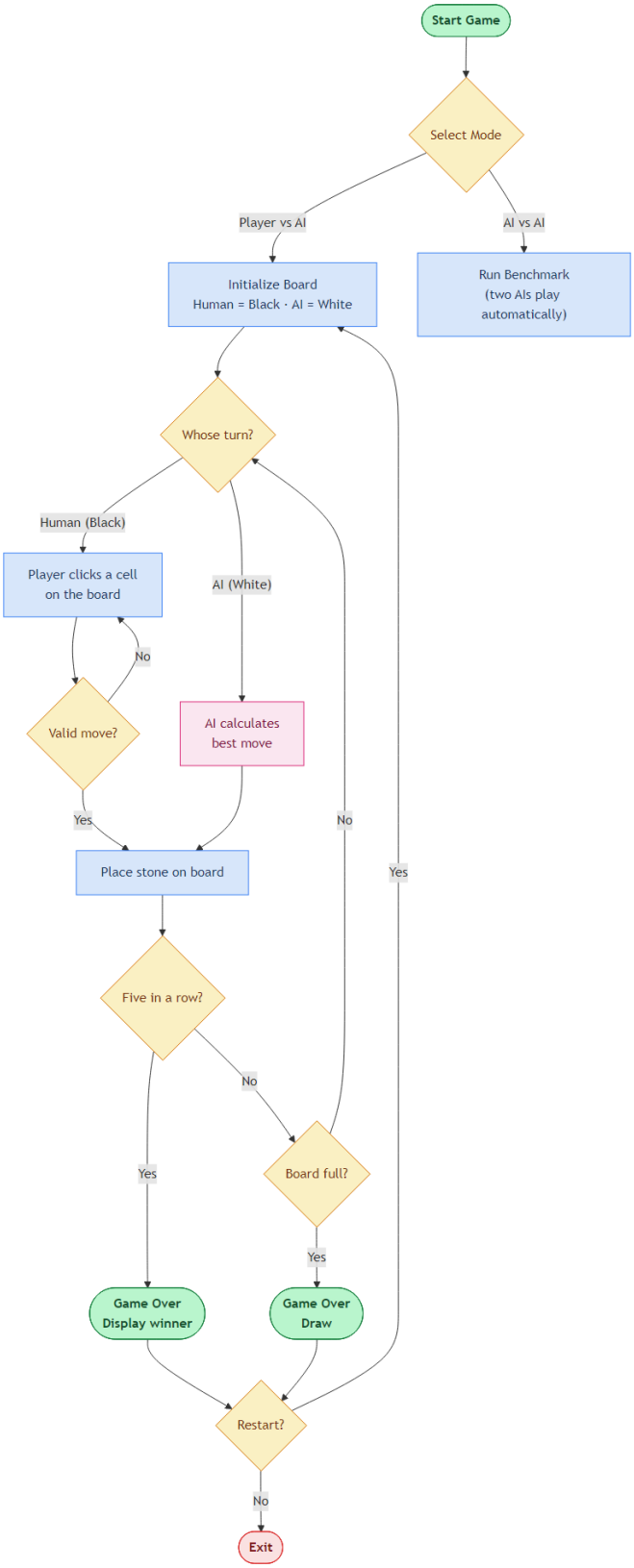
Start Game
Select Mode
Player vs AI
AI vs AI
Initialize Board
Human = Black · AI = White
Run Benchmark
(two AIs play
automatically)
Whose turn?
Human (Black)
AI (White)
Player clicks a cell
on the board
No
Valid move?
AI calculates
best move
Yes
No
Yes
Place stone on board
Five in a row?
No
Yes
Board full?
Yes
Game Over
Display winner
Game Over
Draw
Restart?
No
Exit

Figure 1: Overall game flow of the Gomoku agent. The system supports two operation modes: Player vs. AI and AI vs. AI benchmark. In Player vs. AI mode, the human player (Black) and the AI (White) alternate turns. After each stone placement the game checks for a five-in-a-row win condition or a full-board draw; it then offers the option to restart or exit. In AI vs. AI mode two agents play automatically, which is used for benchmarking search depth and pruning efficiency.

Appendix B: Figure 2 – Stage 1: Candidate Move Generation

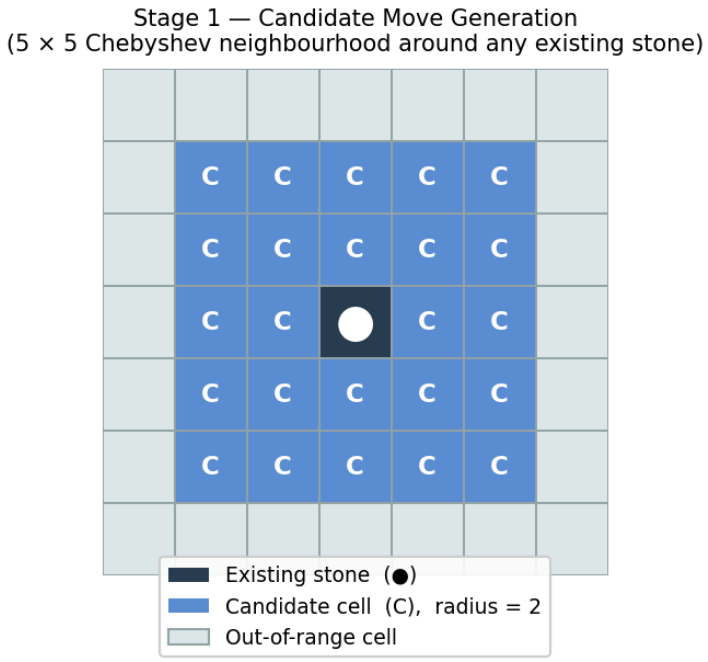

Figure 2: Stage 1 – Candidate move generation: empty cells within the radius-2 Chebyshev neighbourhood of existing stones are selected as candidates.

Appendix C: Figure 3 – Stage 2: Move Ordering

Stage 2 — Move Ordering Priorities

Priority	Category	Condition
1	Winning moves	Places the AI's 5th stone in a row
2	Blocking moves	Prevents the opponent's 5th stone in a row
3	TT move	Best move cached in the transposition table
4	Killer moves	Caused a β-cutoff at the same depth level
5	History rest	Sorted by accumulated cutoff score (desc.)

Figure 3: Stage 2 – Move-ordering priority table: immediate-win moves → blocking moves → transposition-table hit → killer moves → history-heuristic score.

Appendix D: Figure 4 – Stage 3: Alpha-Beta Search

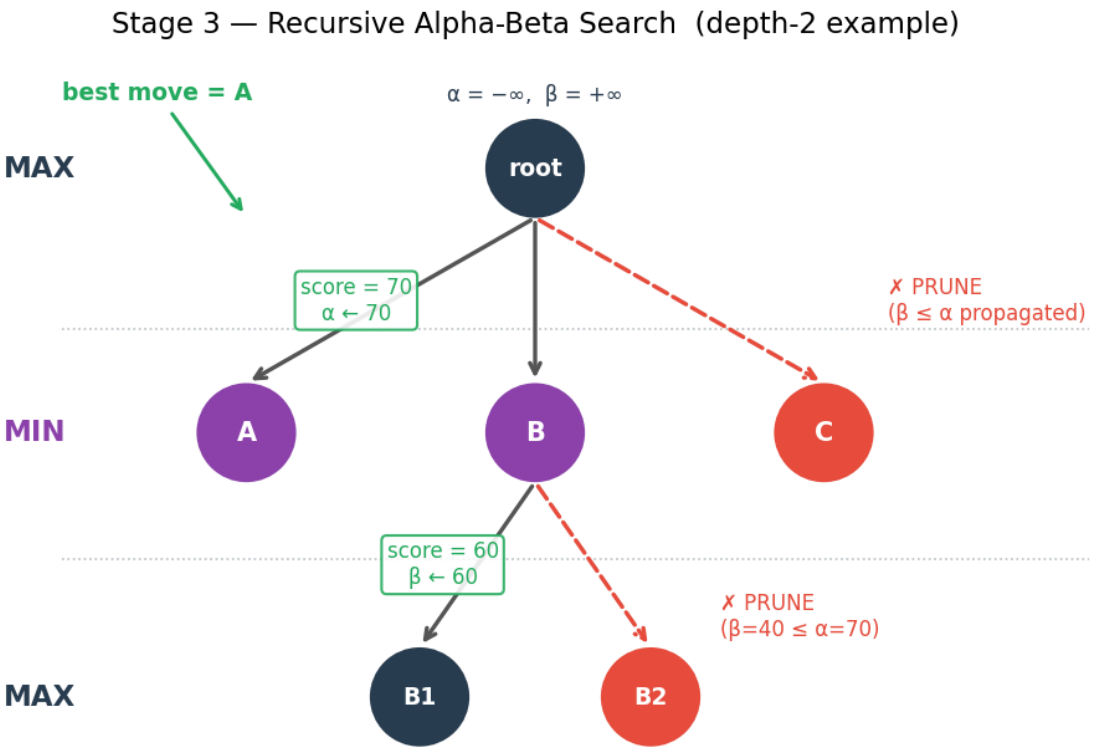

Figure 4: Stage 3 – Alpha-beta search tree with pruning, immediate-win detection, and killer/history move tracking.

Appendix E: Figure 5 – Stage 4: Heuristic Evaluation

Stage 4 — Heuristic Evaluation: Pattern Weights (single direction)

Pattern	Example (x=AI _=empty o=boundary)	Weight
Five	o x x x x x o	1,000,000
Open Four	_ x x x x _	100,000
Rush Four	x x x x _ o / o x x _ x x o	8,000
Open Three	_ x x x _	3,000
Sleep Three	x x x _ _ o	400
Open Two	_ _ x x _ _	100
Sleep Two	x x _ _ _ o	15
— Combined-threat bonus —		
Open Four (any)		+80,000
Rush Four × 2		+60,000
Rush Four + Open Three	(four-three kill)	+50,000
Open Three × 2		+10,000

Figure 5: Stage 4 – Heuristic pattern weights and the asymmetric leaf evaluation: Score = score(AI) − 1.1 × score(opponent).

Appendix F: Figure 6 – AI Decision-Making Workflow

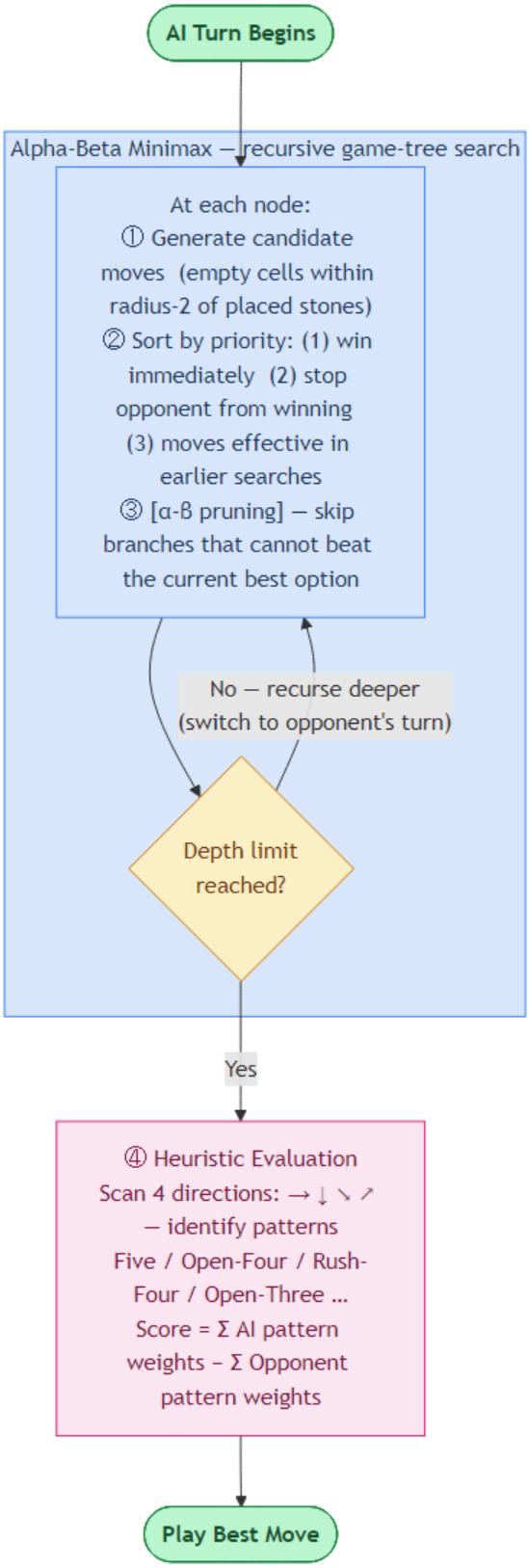

Figure 6: Workflow of the AI's decision-making process using Alpha-Beta Minimax search.

Appendix G: Figure 7 – Decision Time vs. Search Depth

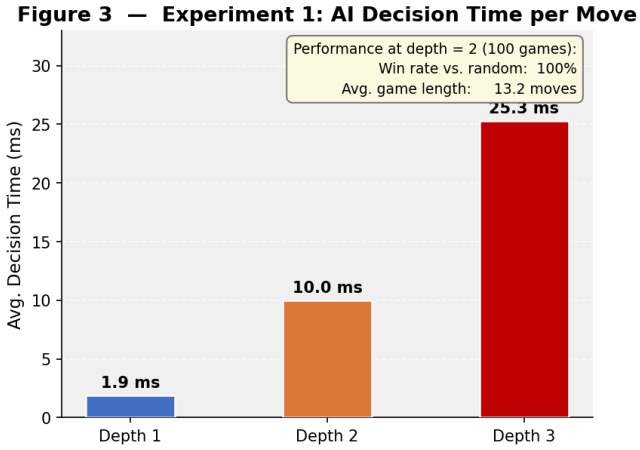

Figure 7: Average AI decision time per move at search depths 1–3 (5 repetitions on a fixed mid-game board position). Log scale is used for the y-axis.

Appendix H: Figure 8 – Minimax vs. Alpha-Beta Node Count

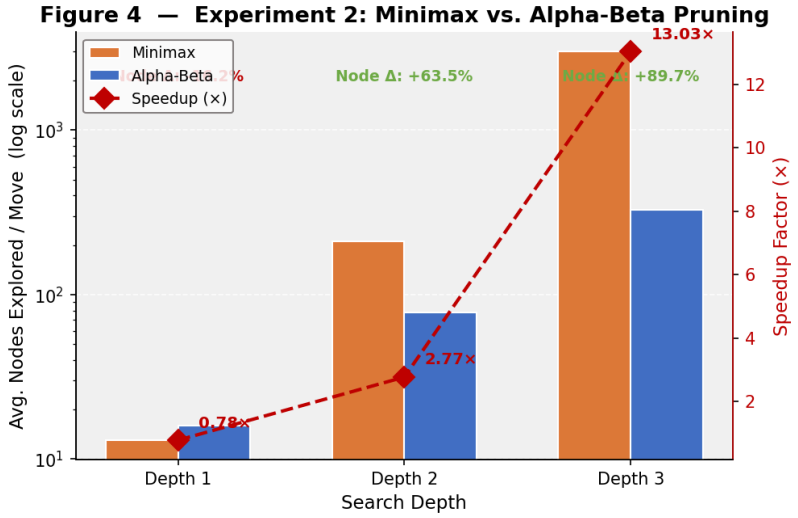

Figure 8: Average nodes explored per move by Minimax (orange bars) and Alpha-Beta (blue bars) at depths 1, 2, and 3. The right panel shows the per-depth speedup factor.

Appendix I: Figure 9 – Win-Rate Heatmap

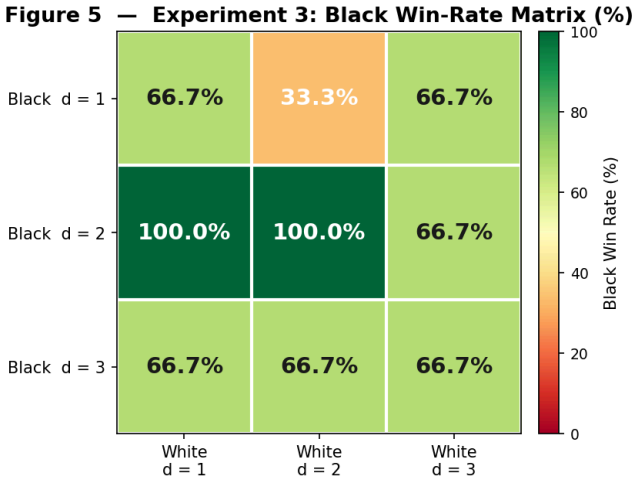

Figure 9: Black win-rate heatmap for the 3×3 depth-matchup tournament (3 games per cell). Each cell shows the fraction of games won by Black; deeper Black search consistently dominates.

Appendix J: Figure 10 – Depth-Scaling Curves

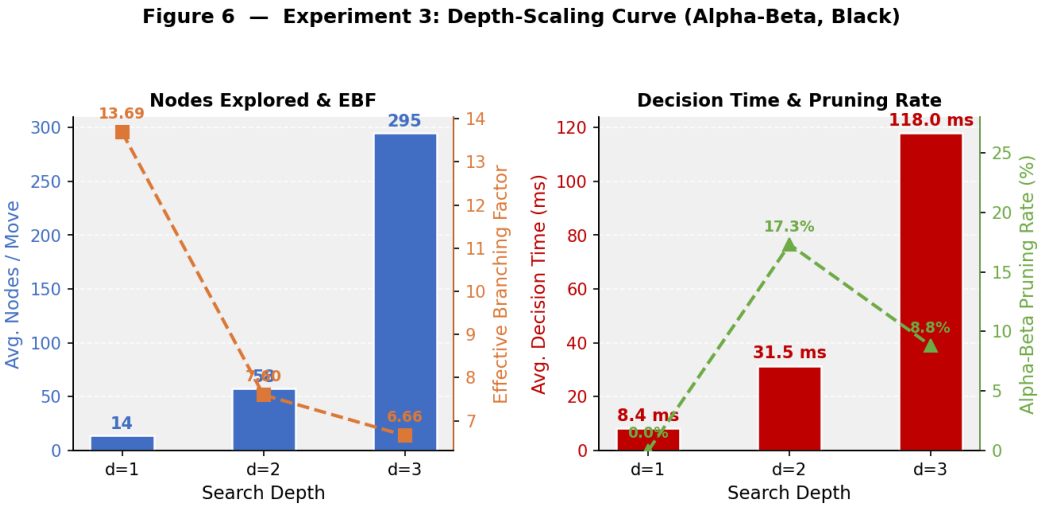

Figure 10: Depth-scaling curves for symmetric same-depth alpha-beta matches (Black perspective, 3 games per depth). Both nodes explored and decision time grow roughly exponentially with depth.